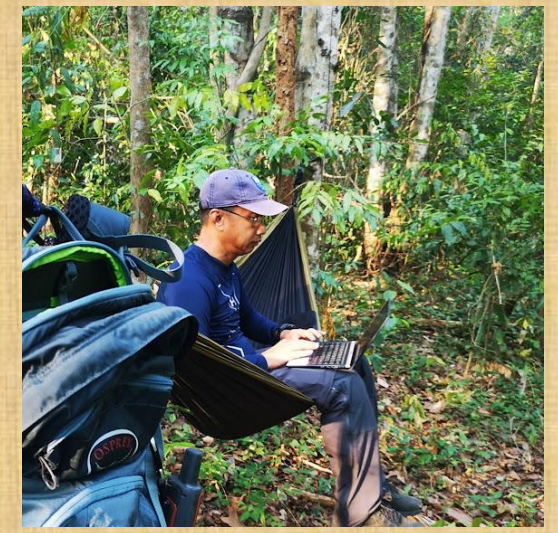
ITCS214 : DATA STRUCTURES & ALGORITHM ANALYSIS

Lecture 6 · Searching Algorithms





found in 2 probes

7	12	19	26	**42**	57	63	78	91

From one-by-one scanning to interval-halving — how programs find things fast

Asst. Prof. Dr. Ananta Srisuphab
ananta.sri@mahidol.ac.th

2026/1

Mahidol University
Faculty of Information and Communication Technology

Learning outcomes

By the end of this lecture, you will be able to…

1 Define the search problem

State search key, search space, and the difference between successful and unsuccessful searches.

2 Implement & trace

Write iterative Java for linear and binary search, and hand-trace low / mid / high on any input.

3 Argue correctness

Use loop invariants (initialization → maintenance → termination) to justify why a search is correct.

4 Analyze cost

Derive best / average / worst-case time and extra space using precise Θ-notation.

5 Extend the toolbox

Explain jump, interpolation, exponential, and Fibonacci search — and their preconditions.

6 Choose wisely

Match an algorithm to the data structure, value distribution, and access pattern of a real problem.

The search problem, precisely

Searching = locating an item in a collection by its key

- **Search key —** the value we compare against (a number, a student ID, a word). Records may carry more data, but the key drives the search.
- **Search space —** the set of positions that could still contain the key. Every algorithm is a strategy for shrinking this set.
- **Successful search —** the key exists; we return its index (or record).
- **Unsuccessful search —** the key is absent; we must *prove* absence and report it (e.g., return −1). Unsuccessful searches are often the expensive case.

Example

Keys: student IDs in an array of 40,000 records.

search(6203114): successful → index 17,204.

search(9999999): unsuccessful → −1, but only after the algorithm has ruled out every remaining position.

Key idea for the whole lecture

The cost of a search is the number of key comparisons needed to shrink the search space to the answer. Faster algorithms shrink it faster — but demand more structure from the data.

Data organization decides the strategy

Two families: scan everything, or repeatedly cut the interval

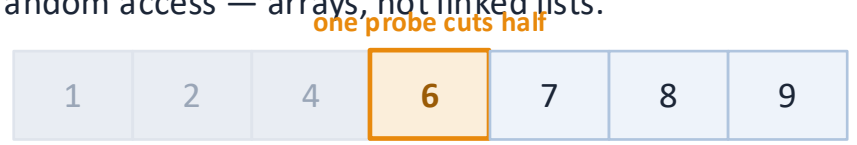
Family 1 — Sequential scanning

Examine candidates one by one. Works on any data: unsorted arrays, linked lists, streams. Each comparison removes exactly one position from the search space.



Family 2 — Interval reduction

Exploit structure (usually sorted order) so that one comparison removes many positions at once. Needs sorted keys and cheap random access — arrays, not linked lists.



Unsorted array / linked list / stream	Sequential scan is the only option
Sorted array (random access)	Interval reduction: binary, jump, interpolation…
Sorted linked list	Random access is Θ(n) → interval tricks don't pay; scan instead

Same question, different data organization → different best algorithm. That trade-off is today's theme.

Linear search: check every candidate

The simplest strategy — and the only one that works on unstructured data

- Scan positions 0, 1, 2, … until the key matches or the data ends.
- No preconditions: sorted or unsorted, array or linked list or stream.
- Each comparison eliminates exactly one position from the search space.
- Return the **index** on a match; return **−1** after the loop proves the key is absent.

Searching for 9 in an unsorted array:



3 comparisons → return 2

Mental model

Reading a shopping receipt line by line to find one item: no shortcuts exist because nothing tells you where the item could be. Gray cells = positions already ruled out.

Iterative Java implementation

LinearSearch.java

```java
public static int linearSearch(int[] arr, int target) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == target) {
            return i;          // successful: index of match
        }
    }
    return -1;                 // unsuccessful: proven absent
}
```

Driver (from the original lecture)

```java
int[] numbers = {3, 5, 1, 4, 2};
int result = linearSearch(numbers, 4);
System.out.println(result != -1
    ? "Found at index: " + result
    : "Not found");            // prints: Found at index: 3
```

- **Inputs:** any int array and a target key.
- **Early exit:** returns as soon as a match is found — best case touches one element.
- **First match wins:** with duplicates, the smallest matching index is returned.
- **–1 is a certificate:** the loop only ends after every position was compared.

Worked trace: find 4 in {3, 5, 1, 4, 2}

The example from the original lecture, step by step



arr[0] = 3 ≠ 4 → eliminate index 0

arr[1] = 5 ≠ 4 → eliminate index 1

arr[2] = 1 ≠ 4 → eliminate index 2

arr[3] = 4 ✓ → return 3

4 comparisons of arr[i] == target were needed. If the target had been 8, the same loop would use 5 comparisons and return −1 — the unsuccessful search always costs n.

Why is it correct? The loop invariant

A property that is true before every iteration — our proof tool for the whole lecture

Invariant

At the start of each iteration with index i: target does not occur in arr[0 .. i−1].

1. Initialization

Before the first iteration, i = 0, so arr[0 .. −1] is empty. The target trivially does not occur in an empty range. ✓

2. Maintenance

If the invariant holds and arr[i] ≠ target, then the target is also absent from arr[0 .. i]. Incrementing i preserves the invariant. ✓

3. Termination

Either arr[i] == target — the returned index is correct — or i reaches n and the invariant says the target is absent from all of arr[0 .. n−1], so −1 is correct. ✓

Correctness intuition: gray cells never need re-checking — the invariant is exactly the statement that the gray region is target-free.

Cost of linear search

Count comparisons of arr[i] == target as a function of n

Case	Comparisons	Θ	When it happens
Best	1	Θ(1)	Target sits at index 0
Average (successful)	≈ (n + 1) / 2	Θ(n)	Target equally likely at each of the n positions
Worst	n	Θ(n)	Target at the last index — or absent

Extra space: Θ(1)

Only the loop counter i lives outside the input. The cost is independent of n — linear search is fully in-place.

- **Why average ≈ (n+1)/2:** if each position is equally likely, the expected index is (1 + 2 + ⋯ + n)/n = (n+1)/2. Halving n does not change the growth rate, so the average case is still Θ(n).
- **Why we write Θ, not O:** Θ(n) says the worst case is linear and no better — a tight bound. O(n) alone would also be (uselessly) true of an algorithm that always finishes in 1 step.

When is linear search the right choice?

Θ(n) is not an insult — sometimes it is optimal

Unsorted or unstructured data

With no order to exploit, every correct algorithm must be able to look everywhere. A single search in unsorted data cannot beat Θ(n).

Sequential-only access

Linked lists, iterators, tape/stream input: you cannot jump to the middle, so interval-reduction methods lose their advantage.

Tiny n

For a handful of elements, the simple loop beats cleverer algorithms in real time — low constant factors, cache-friendly, no setup.

Search once, then throw away

Sorting first costs Θ(n log n) — more than the Θ(n) scan it would replace (details on slide 26).

Rule of thumb: reach for linear search when the data has no exploitable structure — or when building that structure costs more than it saves.

Binary search: halve the interval

One comparison now eliminates half of all remaining candidates

- **Precondition 1 — sorted keys** (ascending here). Order is what lets one comparison speak for many elements.
- **Precondition 2 — cheap random access.** Jumping to the middle must cost O(1): true for arrays, false for linked lists.
- **Strategy:** keep an interval [low, high] that must contain the target if it exists. Probe the middle element:
  - arr[mid] == target → done.
  - arr[mid] < target → target can only be right of mid → low = mid + 1.
  - arr[mid] > target → target can only be left of mid → high = mid − 1.
- Stop when found, or when low > high — the interval is empty, so the key is absent.

Phone-book intuition

Looking up a name, you open near the middle, decide 'earlier or later', and discard half the book — repeatedly. Twenty such cuts handle a million pages.

Halving is powerful

n = 1,000,000 → ≤ 20 probes

n = 1,000,000,000 → ≤ 31 probes

$\lfloor \log_2 n \rfloor + 1$ probes suffice.

Iterative Java — the safe version

BinarySearch.java

```java
public static int binarySearch(int[] arr, int target) {
    int low  = 0;
    int high = arr.length - 1;

    while (low <= high) {                  // interval non-empty
        int mid = low + (high - low) / 2; // overflow-safe

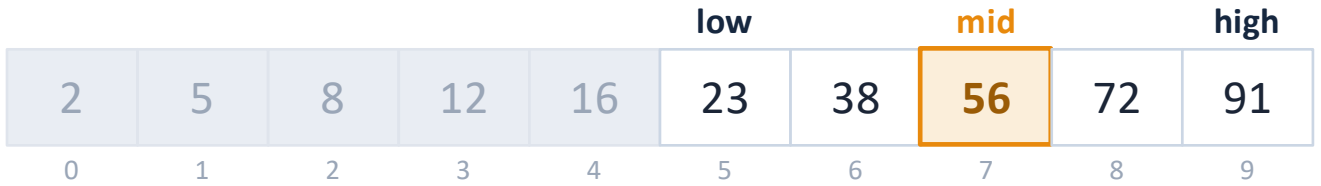
        if (arr[mid] == target) {
            return mid;                    // successful search
        } else if (arr[mid] < target) {
            low = mid + 1;                 // drop left half + mid
        } else {
            high = mid - 1;                // drop right half + mid
        }
    }
    return -1;                            // interval empty: absent
}
```

- **`mid = low + (high − low) / 2`** — same value as (low+high)/2, but immune to int overflow (slide 17).
- **±1 matters:** mid was just compared, so it must leave the interval. Keeping it risks an infinite loop.
- **Empty array?** high = −1 < low = 0, the loop body never runs, −1 is returned — correct with no special case.

Successful trace: find 23 (n = 10)



1. mid = 0 + (9−0)/2 = 4. arr[4] = 16 < 23 → low = 5



2. mid = 5 + (9−5)/2 = 7. arr[7] = 56 > 23 → high = 6

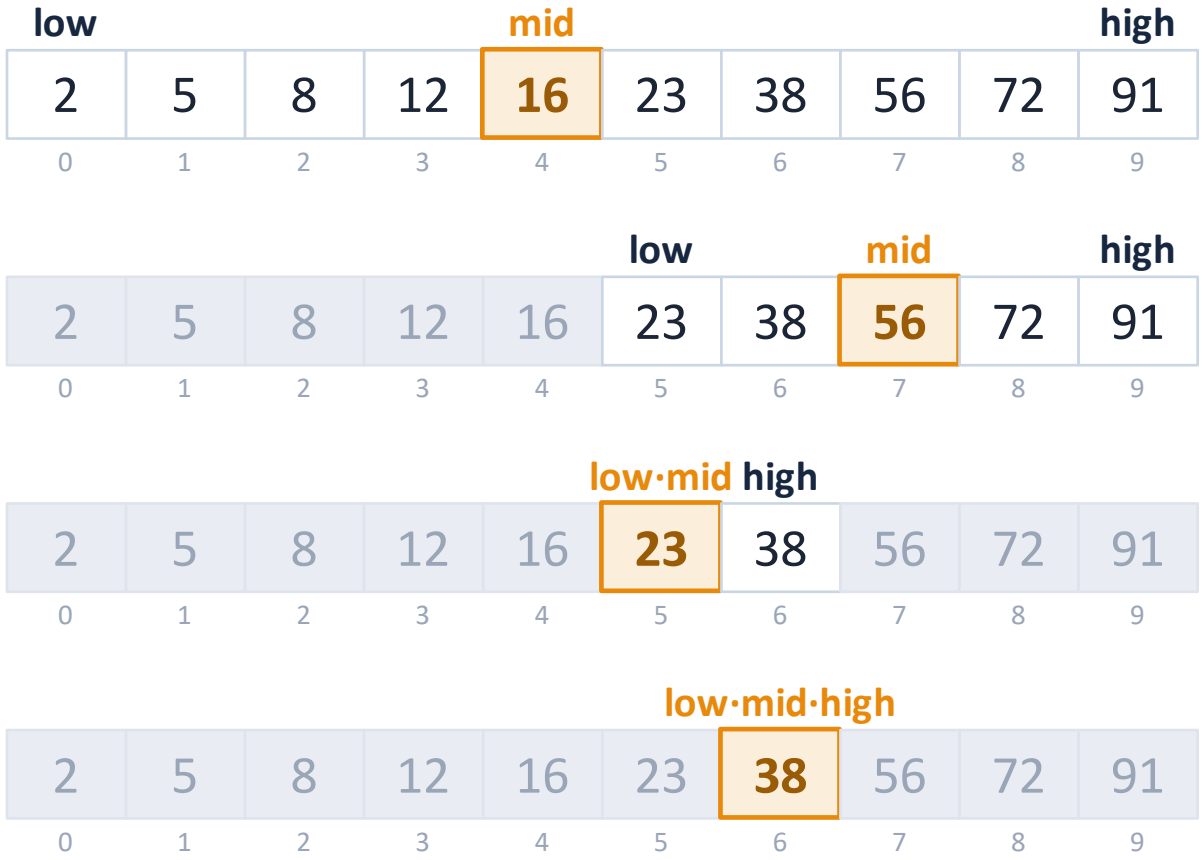


3. mid = 5 + (6−5)/2 = 5. arr[5] = 23 ✓ → return 5

3 probes instead of 6 for a linear scan — and the gap widens fast as n grows.

Unsuccessful trace: find 40

How the algorithm proves a key is absent



1. arr[4] = 16 < 40 → low = 5
2. arr[7] = 56 > 40 → high = 6
3. arr[5] = 23 < 40 → low = 6
4. arr[6] = 38 < 40 → low = 7

Now low = 7 > high = 6: the interval is empty. Every position is gray → return −1 after only 4 probes.

Two classic pitfalls

Both have shipped in real production libraries

Pitfall 1 — while (low < high)

Using < instead of <= skips the one-element interval low == high. A target sitting exactly there is reported absent. Rule: loop while the interval [low, high] is non-empty, i.e. low ≤ high.

Pitfall 2 — mid = (low + high) / 2

For low, high near Integer.MAX_VALUE the sum overflows to a negative int, mid becomes negative, and arr[mid] throws. This bug sat in the JDK's own binarySearch for nine years.

```
The fix — mathematically identical, overflow-proof
int mid = (low + high) / 2;          // ✗ can overflow
int mid = low + (high - low) / 2;    // ✓ high - low fits in int
int mid = (low + high) >>> 1;        // ✓ unsigned-shift trick
```

Why it matters

Binary search is 'simple', yet the first published binary search (1946) predates the first bug-free version (1962). Details hide in intervals and indices.

Correctness of binary search

Same invariant framework — now guarding both sides of the interval

Invariant

Before every loop test: if target occurs in arr, then its index lies in [low, high].

1. Initialization

low = 0, high = n − 1: the interval is the whole array, which certainly contains the target's index if it exists. ✓

2. Maintenance

Sortedness is used here. If arr[mid] < target, every index ≤ mid holds a value ≤ arr[mid] < target — so setting low = mid + 1 discards only impossible positions. Symmetrically for high = mid − 1. ✓

3. Termination

high − low shrinks by at least 1 per iteration (mid always leaves the interval), so the loop ends. Exit with a match → index correct. Exit with low > high → interval empty, invariant says target cannot occur → −1 correct. ✓

The ±1 updates do double duty: they keep the invariant true and force termination. That is why 'low = mid' bugs loop forever.

Deriving Θ(log n)

How many halvings until one candidate remains?

After 0 probes: n candidates

After 1 probe: ≤ n / 2

After 2 probes: ≤ n / 4

After k probes: ≤ $n / 2^k$

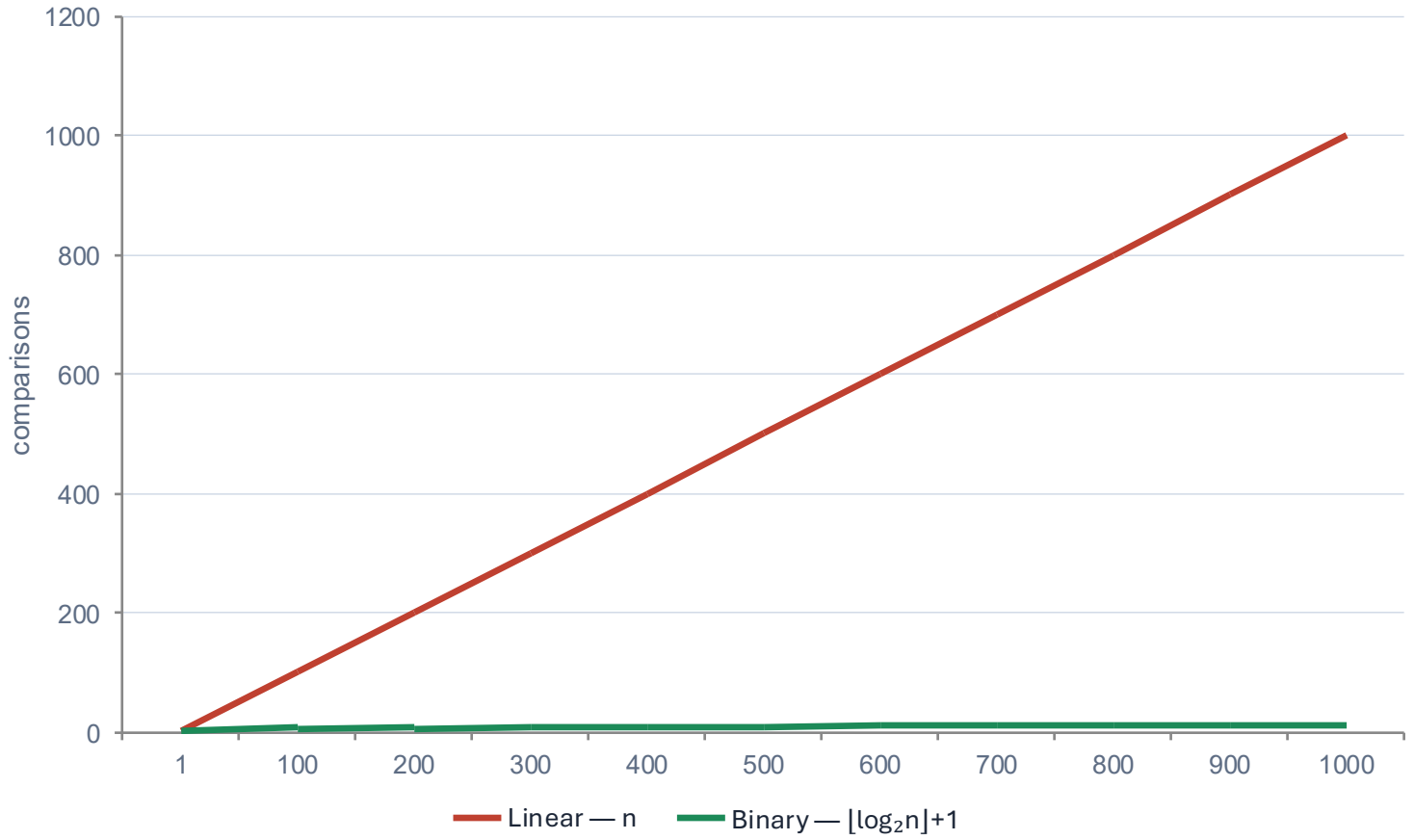
- **Stop condition:** the loop can continue only while $n / 2^k \geq 1$.
- **Solve:** $n / 2^k = 1 \implies 2^k = n \implies k = \log_2 n$.
- So at most $\lfloor \log_2 n \rfloor + 1$ probes — worst case Θ(log n), for both successful and unsuccessful searches.
- **Best case Θ(1):** the very first mid is the target.
- **Average case Θ(log n):** most nodes of the implicit decision tree sit near the bottom, so a random successful search still costs ≈ $\log_2 n - 1$ probes.

Space

Iterative version: Θ(1) extra space — just low, high, mid. A recursive version makes one call per halving, so its call stack holds Θ(log n) frames. Same time, different space.

Θ(n) vs Θ(log n), drawn to scale

Worst-case comparisons as n grows



Read the green line

At this scale binary search hugs the x-axis: 1,000 elements need at most 10 probes. Doubling n adds one probe; doubling n doubles the red line.

Big numbers

$n = 10^6 \rightarrow \leq 20$ probes

$n = 10^9 \rightarrow \leq 31$ probes

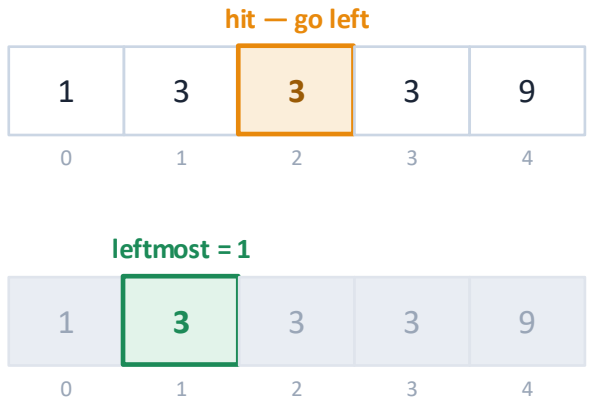
Every atom in 10^9: still a screenful of probes.

Variant: first occurrence

*With duplicates, plain binary search returns *some* match — often you need the leftmost*

```
firstOccurrence — keep going left after a match
public static int firstOccurrence(int[] arr, int target) {
    int low = 0, high = arr.length - 1, result = -1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == target) {
            result = mid;        // remember this hit…
            high   = mid - 1;    // …but keep searching LEFT
        } else if (arr[mid] < target) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return result;
}
```

arr = {1, 3, 3, 3, 9}, target 3:



Still Θ(log n): each iteration halves the interval — a match no longer stops the loop, it just narrows the goal.

Variants: last occurrence & insertion point

```
lastOccurrence — mirror image
if (arr[mid] == target) {
    result = mid;        // remember…
    low = mid + 1;       // …keep searching RIGHT
} else if (arr[mid] < target) { low = mid + 1; }
else                          { high = mid - 1; }
```

```
insertionPoint — where target WOULD go
while (low <= high) {
    int mid = low + (high - low) / 2;
    if (arr[mid] < target) low = mid + 1;
    else                   high = mid - 1;
}
return low;  // first index with arr[i] >= target
```

- **Insertion point** = where an absent key belongs, keeping the array sorted. On failure, low already holds it — the loop drove low to the first element ≥ target.
- **Java's contract:** Arrays.binarySearch returns `-(insertionPoint) - 1` for absent keys — one int encodes both 'not found' and 'insert here'.
- **Count duplicates** in Θ(log n): lastOccurrence − firstOccurrence + 1.

Edge cases: duplicates & empty arrays

Where hand-written binary searches actually fail

Empty array (n = 0)

high = −1 < low = 0: the safe version returns −1 immediately. Versions that probe arr[mid] before checking the interval throw on arr[0].

Single element (n = 1)

low = high = 0 — the loop MUST still run once. This is exactly the case the low < high pitfall breaks.

Duplicates

Plain binary search may return ANY matching index — for {1,3,3,3,9} and target 3 it returns 2, not 1. Use firstOccurrence / lastOccurrence when position among equals matters.

Unsorted input

Binary search does not detect disorder — it just returns wrong answers with full confidence. Garbage in, confidently wrong out: the precondition is the caller's contract.

These four cases are precisely the JUnit tests we write in section 08.

Is binary search automatically the best?

Only if someone already paid for the sorted order

One search in UNSORTED data: linear scan Θ(n) vs sort Θ(n log n) + binary search Θ(log n)

	Linear search once	Sort, then binary search once
Work for n = 1,000,000	≈ 10^6 comparisons	≈ n $\log_2$ n ≈ 2 × 10^7 (sort) + 20 (search)
Total growth	Θ(n)	Θ(n log n)
Winner for a single search	✓ linear — by a factor of ≈ log n	✗ sorting dominates everything

- **Sorting is an investment,** not a free precondition. Θ(n log n) up front buys Θ(log n) per later search.
- **Already sorted?** Then the investment was made earlier (or the data arrives ordered) — binary search wins immediately.

When does sorting pay off? Repeated searches

Compare k searches under both plans

Plan A — stay unsorted

k linear searches:

cost ≈ k · n

Plan B — sort once, then binary

one sort + k binary searches:

cost ≈ n $\log_2$ n + k · $\log_2$ n

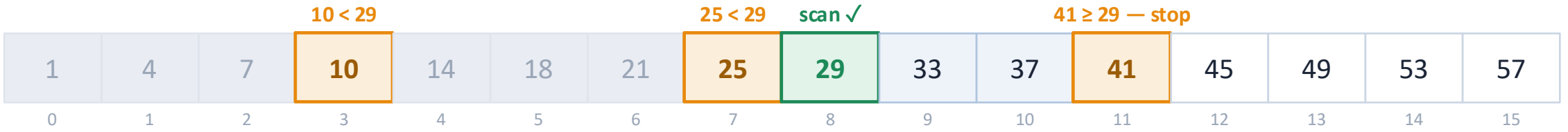
- **Break-even:** Plan B wins when n log n + k log n < k n, i.e. roughly when **k > $\log_2$ n** (for large n). For n = 10^6 that is only ≈ 20 searches!
- **Amortized view:** spread over k searches, the sort adds (n log n)/k per search — negligible once k is large.
- **Real systems generalize this:** databases build an index (a sorted structure) once, precisely because they expect millions of subsequent lookups.
- **Dynamic data caveat:** if items keep arriving, re-sorting per insert is Θ(n) — balanced BSTs or B-trees (next lectures) keep both insert and search at Θ(log n).

Jump search: probe in √n-sized strides

Precondition: sorted array with random access

- **Idea:** leap ahead [√n] positions at a time until a probe meets a value ≥ target, then linear-scan just the last block.
- **Why √n strides?** Cost ≈ n/m jumps + m scan steps; minimizing over m gives m = √n → total Θ(√n).
- **Only jumps backwards once** — useful when moving backward is expensive (e.g., sequential media) but skipping forward is cheap.

n = 16, step = 4, target 29 — probes at indices 3, 7, 11, then scan block 8…11:



3 jumps + 1 scan step = 4 probes. Linear: 9. Binary: 4 — but jump search never revisits earlier blocks.

Jump search: Java & analysis

JumpSearch.java

```java
public static int jumpSearch(int[] arr, int target) {
    int n = arr.length;
    if (n == 0) return -1;
    int step = (int) Math.sqrt(n);
    int prev = 0;
    while (arr[Math.min(step, n) - 1] < target) {
        prev = step;              // this block is too small
        step += (int) Math.sqrt(n);
        if (prev >= n) return -1;   // ran past the end
    }
    for (int i = prev; i < Math.min(step, n); i++) {
        if (arr[i] == target) return i;   // scan one block
    }
    return -1;
}
```

Time

Θ(√n) worst case: ≤ ⌈n/√n⌉ jumps + ≤ √n scan steps. Between Θ(log n) and Θ(n).

Space

Θ(1) — three ints.

Wins when

Jumping back is costly, or comparisons at far-apart indices are expensive (disk/tape, memory-mapped data).

Loses when

Full random access is cheap — binary search's Θ(log n) dominates Θ(√n) for all large n.

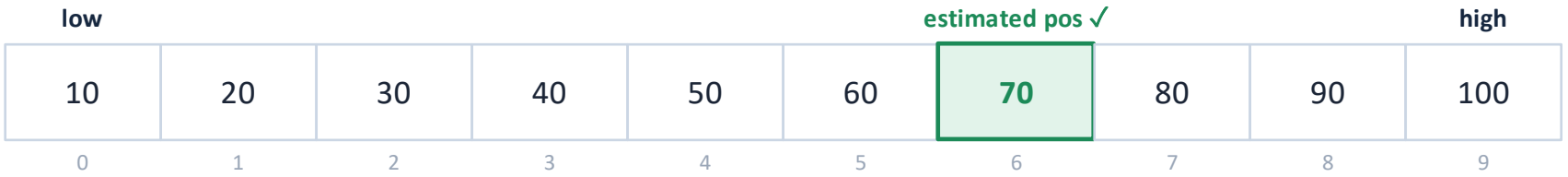
Interpolation search: probe where the value should be

Preconditions: sorted array, random access — and roughly uniform values

- **Idea:** a phone book user doesn't open to the middle for 'Zhang' — they open near the end. Estimate the target's position from its value:

```
pos = low + (target − arr[low]) · (high − low) / (arr[high] − arr[low])
```

Uniform data {10, 20, 30, …, 100}, target 70: pos = 0 + (70−10)·(9−0)/(100−10) = 6 → arr[6] = 70 ✓ first probe!



The catch

The estimate is only good if values grow ≈ linearly with index. On skewed data — {1, 2, 3, …, 9, 10^9} — the probes cluster uselessly and the search degrades to Θ(n).

Interpolation search: Java & analysis

InterpolationSearch.java

```java
public static int interpolationSearch(int[] arr, int target) {
    int low = 0, high = arr.length - 1;
    while (low <= high && target >= arr[low]
                       && target <= arr[high]) {
        if (arr[high] == arr[low])              // all equal:
            return (arr[low] == target) ? low : -1;
        int pos = low + (int) ((long) (target - arr[low])
                * (high - low) / (arr[high] - arr[low]));
        if      (arr[pos] == target) return pos;
        else if (arr[pos] <  target) low  = pos + 1;
        else                         high = pos - 1;
    }
    return -1;
}
```

Expected time

Θ(log log n) — for keys drawn independently from a (near-)uniform distribution. For n = 10^9: ≈ 5 probes.

Worst case

Θ(n) on adversarial / highly skewed values.

Space

Θ(1). Note the (long) cast: the numerator can overflow int.

Range guard

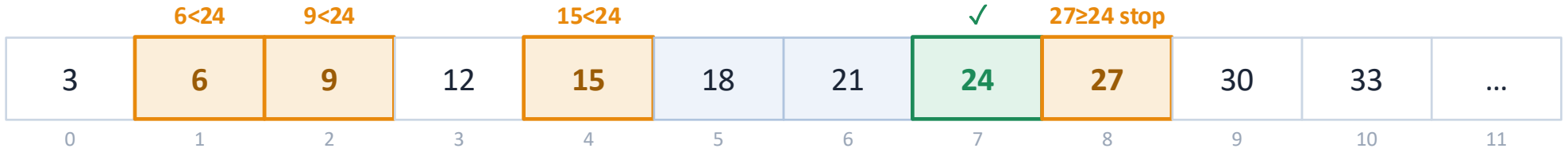
target must lie within [arr[low], arr[high]] or the formula extrapolates outside the array.

Exponential search: find the range, then go binary

Precondition: sorted data — but the size may be unknown or unbounded

- **Idea:** probe indices 1, 2, 4, 8, 16, … until arr[bound] ≥ target. The target must lie in (bound/2, bound] — binary search only that slice.
- **Killer feature:** never needs n up front — perfect for unbounded/streaming sorted data, linked storage with unknown length, or 'infinite' arrays in interview problems.

Target 24 — doubling probes 1, 2, 4, 8, then binary search on indices 5…8:



4 doubling probes + 2 binary probes. If the target sits at index i, the bound stops after $\lceil \log_2 i \rceil + 1$ probes — cost depends on i, not on n.

Exponential search: Java & analysis

ExponentialSearch.java

```java
public static int exponentialSearch(int[] arr, int target) {
    if (arr.length == 0) return -1;
    if (arr[0] == target) return 0;
    int bound = 1;
    while (bound < arr.length && arr[bound] < target) {
        bound *= 2;                    // 1, 2, 4, 8, ...
    }
    int low  = bound / 2 + 1;
    int high = Math.min(bound, arr.length - 1);
    return binarySearch(arr, target, low, high);
}   // uses a ranged binarySearch(arr, t, low, high)
```

Time

Θ(log i) for a target at index i — 'output-sensitive'. For a finite array, worst case Θ(log n).

Space

Θ(1) with an iterative ranged binary search.

Use when

Size is unknown/unbounded, or targets cluster near the front (log i ≪ log n).

Doubling phase ≈ $\log_2 i$ probes + binary phase over an interval of length ≤ i/2 ≈ $\log_2 i$ more → Θ(log i) total. When i ≪ n (target near the front), this beats plain binary search.

Fibonacci search: split by golden-ratio blocks

Precondition: sorted array with random access

- **Idea:** cover the array with the smallest Fibonacci number F(k) ≥ n. Probe at offset F(k−2); the two possible remainders have sizes F(k−1) and F(k−2) — both Fibonacci again, so the scheme recurses cleanly.
- **Instead of halving** (½, ½), each probe splits the interval ≈ (0.382, 0.618) — the golden ratio.
- **Historical niche:** needs only addition and subtraction to move probes (no division, no midpoint) — attractive on early hardware, and when probing nearby indices is cheaper (tape, sequential storage).

n = 12 → F(7) = 13 covers it. First probe at index min(F(5), n−1) = 5, splitting 12 cells into 5 + 7:



Fibonacci sizes: 1, 1, 2, 3, 5, 8, 13, 21, … each interval shrinks by factor ≈ 0.618 per probe → still logarithmic.

Fibonacci search: Java & analysis

FibonacciSearch.java

```java
public static int fibonacciSearch(int[] arr, int target) {
    int n = arr.length;
    int fibA = 0, fibB = 1, fib = fibA + fibB; // F(k-2),F(k-1),F(k)
    while (fib < n) { fibA = fibB; fibB = fib; fib = fibA + fibB; }
    int offset = -1;                // everything <= offset is ruled out
    while (fib > 1) {
        int i = Math.min(offset + fibA, n - 1);   // probe index
        if (arr[i] < target) {          // drop left F(k-2) block
            fib = fibB; fibB = fibA; fibA = fib - fibB;
            offset = i;
        } else if (arr[i] > target) {   // drop right F(k-1) block
            fib = fibA; fibB = fibB - fibA; fibA = fib - fibB;
        } else return i;
    }
    if (fibB == 1 && offset + 1 < n && arr[offset + 1] == target)
        return offset + 1;          // last remaining candidate
    return -1;
}
```

Time

Θ(log n) — interval shrinks by ≈ 0.618 each probe; ≈ $1.44 \cdot \log_2 n$ probes worst case.

Space

Θ(1) — three Fibonacci registers + offset.

Trade-off

Slightly more probes than binary search, but probe positions move by additions only, in one direction at a time.

Six algorithms, one table

Algorithm	Prerequisites	Strategy	Expected	Worst	Extra space
Linear	none (any data, any container)	scan one by one	Θ(n)	Θ(n)	Θ(1)
Binary	sorted + random access	halve the interval	Θ(log n)	Θ(log n)	Θ(1) iter · Θ(log n) rec
Jump	sorted + random access	√n strides, then scan	Θ(√n)	Θ(√n)	Θ(1)
Interpolation	sorted + ≈ uniform values	probe at estimated position	Θ(log log n)*	Θ(n)	Θ(1)
Exponential	sorted; size may be unknown	double bound, then binary	Θ(log i)	Θ(log n)	Θ(1)
Fibonacci	sorted + random access	golden-ratio splits	Θ(log n)	Θ(log n)	Θ(1)

** expected Θ(log log n) only under (near-)uniformly distributed keys; i = index of the target (exponential search is output-sensitive).*

What actually powers large systems

Binary search is an idea inside these systems — rarely the whole system

Domain	The workhorse structure	Where our lecture's ideas appear
Database indexes	B-trees / B+-trees: wide, shallow sorted trees tuned to disk pages	Binary (or linear) search inside each node; the tree, not the array, gives Θ(log n) I/O
Hash indexes / hash maps	Hashing — expected Θ(1) lookups, no order at all	No binary search involved; the counterpoint to comparison-based searching
Search engines	Inverted indexes: term → sorted posting lists	Binary search / galloping when intersecting posting lists
File systems	Directory structures such as B-trees (NTFS, ext4 htree, APFS)	Sorted-structure lookups on metadata, not one big sorted array
Bioinformatics	Suffix arrays, BWT/FM-index (e.g. read aligners like BWA)	Binary search over suffix arrays is one component among specialized algorithms

Honest framing: 'databases use binary search' is a half-truth. They use sorted structures because sortedness enables logarithmic search — the principle survives even where the plain algorithm doesn't.

Where you will really meet binary search

As a component — often on things that aren't arrays at all

Standard libraries

Java Arrays.binarySearch / Collections.binarySearch; C++ lower_bound / upper_bound — the first/last-occurrence variants from slides 22–23, productionized.

Binary search on the answer

git bisect finds the commit that broke the build in Θ(log #commits). Same trick: minimize capacity/speed/threshold where a yes/no test is monotone.

Numeric root finding

Bisection on a continuous monotone function: each step halves an interval of numbers instead of indices.

Inside other algorithms

Timsort's galloping merge (exponential + binary), posting-list intersection, percentile lookups in sorted summaries.

Transferable insight: any monotone yes/no question over an ordered domain can be answered in Θ(log) probes.

SUMMARY

Structure buys speed

Unstructured data forces Θ(n) scanning; sorted + random access unlocks Θ(log n) halving — and distribution knowledge even Θ(log log n).

Preconditions have prices

Sorting costs Θ(n log n); it pays once you search more than ≈ log n times. Count total work.

Correctness is an invariant

Initialization → maintenance → termination. The ±1 updates keep the invariant AND force progress.

Details bite

low ≤ high, overflow-safe mid, empty arrays, duplicates — test first/middle/last, absent, empty, single.

Closing challenge (next class): an array was sorted, then rotated at an unknown pivot — e.g. {38, 56, 72, 91, 2, 5, 8, 12}. Design a Θ(log n) search. Hint: one half is always still sorted; which comparison tells you which?